

Allow `[[nodiscard]]` in type alias declarations

Document #:	P3245R0
Date:	2024-04-16
Project:	Programming Language C++
Audience:	Evolution Working Group Incubator
Reply-to:	Xavier Bonaventura (BMW)

§ 1 Abstract

This paper proposes to allow the usage of `[[nodiscard]]` to be used in type alias declarations in the same way that can be done with `[[deprecated]]`.

§ 2 Current usage of `[[nodiscard]]` in types

`[[nodiscard]]` was initially proposed in [\[P0068R0\]](#) and [\[P0189R1\]](#) to be used in types and functions. For types, it can be used in the following way:

```
struct [[nodiscard]] Error{
    int code;
};

MyError critical_call(){
    return {42};
}

int main(){
    // Most compilers will issue diagnostics on the next call
    // for any function that returns the MyError type
    critical_call();
}
```

This allows users to notify the implementation that any function returning a type `Error`, its return value should not be discarded silently.

§ 3 The missing use case

There is one use case that is missing with the current allowed usage of the `[[nodiscard]]` attribute that would be useful.

Imagine an external library that defines a type `Error` that we want to use in our library. Because this is conceptually an error type, in a safety critical system you might be interested on having this type treated as `[[nodiscard]]`.

If the library already marks the type as `[[nodiscard]]` then everything is good and you can use it directly or using an alias. If that is not the case, then it is not so simple.

Intuitively we would put the `[[nodiscard]]` attribute at the same place like it is allowed nowadays with `[[deprecated]]`.

```
using MyError [[nodiscard]] = Error;
```

However, the attribute above will be ignored. The grammar allows the attribute to be in such a position but it is not allowed by the `[[nodiscard]]` attribute specification 9.12.9 [\[dcl.attrnodiscard\]](#).

§ 4 Proposal

The proposal would be to allow the usage of `[[nodiscard]]` in type aliases. This is already possible in a non-standard way using compiler annotations.

Before	After
<pre>// In clang using MyError [[clang::warn_unused_result]] = Error; // In gcc using MyError __attribute__((warn_unused_result)) = Error; // In MSVC // I'm not aware that is possible</pre>	<pre>using MyError [[nodiscard]] = Error;</pre>

§ 4.1 Problems and potential solutions

The main problem with the proposal above is that an alias declaration is not a type. This is a big difference when trying to apply `[[nodiscard]]` in comparison to `[[deprecated]]`. In the case of `[[deprecated]]`, the compiler can issue a warning in the moment that it sees the name in the declaration. For the case of `[[nodiscard]]`, the compiler would have to keep this information additionally because the alias is not a type.

In this section we will present the two possible solutions that could be done. I would like to get feedback from the committee which one is preferred.

§ 4.1.1 Aliases carry `[[nodiscard]]` information

This option would require to remember if a function was seen with the alias or with the original type. This would mean that if the compiler has seen the function with the return alias, it should issue the warning when the return value is not used.

```
Error foo(int bar);

foo(42); // No warning is issued`

MyError foo(int bar); // Same declaration like above due to Error and
                       // MyError being the same type

foo(42); // `[[nodiscard]] warning is issued`
```

§ 4.1.2 Alias mechanism to introduce a type with different discardability semantics

Another direction could be considered is to introduce a mechanism to introduce a type from another one. In this case, this would mean that `Error` and `MyError` would be two different types, one that is `[[nodiscard]]` and one that is not.

§ 5 Wording

There is no wording yet, first the direction needs to be discussed. Depending on that the work on the wording will be done.

§ 6 Acknowledgements

Thanks a lot to everyone that participated in the initial discussion in Mattermost and during the Tokyo meeting. This allowed me to create the first version of the paper already exposing the difference between `[[nodiscard]]` and `[[deprecated]]` and exposing the challenge with adding `[[nodiscard]]` in type aliases.

§ 7 References

[P0068R0] Andrew Tomazos. 2015-09-03. Proposal of `[[unused]]`, `[[nodiscard]]` and `[[fallthrough]]` attributes.

<https://wg21.link/p0068r0>

[P0189R1] Andrew Tomazos. 2016-02-29. Wording for `[[nodiscard]]` attribute.

<https://wg21.link/p0189r1>